Proyecto 2 - Catálogos temáticos con MongoDB, Netlify Functions y autenticación JWT

Este proyecto es la continuación del **Proyecto 1 (Catálogos temáticos)**. Se parte de la aplicación y el servicio REST ya desarrollados, usando el mismo tema asignado, las mismas tres entidades (una principal y dos secundarias) y los mismos frontends en Vue.js. No se repite aquí la especificación inicial: se mantienen todos sus requisitos, con las adaptaciones que se describen a continuación.

1. Migración de la plataforma FaaS a Netlify

El servicio REST, que en el Proyecto 1 se ejecutaba como FaaS en Cloudflare, debe migrarse a **funciones serverless de Netlify (Netlify Functions)**.

- Cada recurso del servicio REST se implementa como una o más funciones de Netlify, manteniendo las mismas rutas y operaciones del Proyecto 1 (por ejemplo, redirigiendo `/api/*` hacia las funciones mediante `netlify.toml` o `_redirects`).
- El frontend en Vue.js y las funciones se publican juntos en **Netlify**, con despliegue desde el repositorio o mediante Netlify CLI.
- Las variables sensibles (cadena de conexión a MongoDB Atlas y clave de firma JWT) se configuran como **variables de entorno de Netlify**, nunca dentro del código entregado.

2. Migración de la base de datos a MongoDB

El Proyecto 1 debe adaptarse para utilizar **MongoDB Atlas** en lugar de la base de datos relacional (SQLite / D1).

- Cada entidad pasa a ser una **colección** de MongoDB y cada registro un **documento**.
- Se deben conservar los datos del Proyecto 1: al menos 12 documentos de la entidad principal y 6 de cada entidad secundaria, con sus imágenes.
- Las relaciones entre entidades se mantienen mediante referencias (por ejemplo `ObjectId` o identificadores propios) que permitan navegar entre ellas.
- Se debe conservar todo el CRUD del servicio REST (listar, consultar, crear, editar y borrar) con el mismo comportamiento que en el Proyecto 1.
- La conexión a Atlas debe reutilizarse entre invocaciones de la función y el clúster debe permitir el acceso desde Netlify (configuración de acceso de red en Atlas).

3. Autenticación con JWT

Se debe incorporar autenticación al sitio utilizando **JSON Web Tokens (JWT)**, según lo visto en el tutorial de autenticación del curso.

Servicio REST

- Agregar una colección de **usuarios** en MongoDB (por ejemplo: nombre, correo, contraseña con hash y

rol).
- Las contraseñas **nunca** se almacenan en texto plano; deben guardarse con un hash seguro.
- Endpoints de autenticación:
  - Registro de usuario.
  - Inicio de sesión (login): valida credenciales y devuelve un JWT firmado.
- El token debe incluir al menos la identidad del usuario y una fecha de expiración (`exp`), y firmarse con una clave secreta guardada como variable de entorno en Netlify.
- El cliente envía el token en cada solicitud protegida mediante el encabezado `Authorization: Bearer <token>`.
- Cada función protegida debe validar la firma y la expiración del token antes de atender la solicitud, y responder con los códigos HTTP adecuados:
  - `401` si no hay token o es inválido/expirado.
  - `403` si el usuario no tiene permiso para la operación.
- Las operaciones que modifican datos (crear, editar y borrar) deben requerir un usuario autenticado. Las operaciones de consulta (listar y ver detalle) pueden ser públicas o protegidas, según se defina en el diseño.

Frontend (Vue.js)

- Pantallas de registro e inicio de sesión.
- Almacenar el token en el cliente y adjuntarlo automáticamente a las solicitudes al servicio.
- Mostrar u ocultar las acciones de crear, editar y borrar según el usuario esté autenticado.
- Opción de cierre de sesión (logout) que elimina el token.
- Manejar la expiración del token redirigiendo al inicio de sesión cuando el servicio responda `401`.

4. Entrega

- Publicar la aplicación actualizada en **Netlify**.
- Entregar un archivo `.zip` con todo el código generado, incluyendo la carpeta de funciones y el archivo de configuración de Netlify (**no** incluir `node_modules` ni archivos con secretos, como cadenas de conexión, archivos `.env` o claves de firma).
- Incluir un archivo `README.md` con el nombre del estudiante, el enlace donde se publicó el sistema y las instrucciones para configurar las variables de entorno necesarias (conexión a MongoDB Atlas y clave JWT).